Search Before You Prove: Verified Retrieval and Reuse of Shared Lean Libraries in Multi-Agent Autoformalization

Zibo Yang
FAIR at Meta

Agents that formalize mathematics re-prove results that already exist. In a multi-agent setting the waste compounds: two agents state the same lemma twice, a roadmap plans a node that a shared library already proves, and a duplicate that compiles is never flagged by the proof checker. We describe the search-and-reuse mechanism of AutoformBot, which lets agents ask, before they add a node to a blueprint or a helper to a Lean project, whether the project's own blueprint or a Lean library the project depends on already contains the result. The mechanism has four parts: a read-only search over Markdown blueprints; an index format that a Lean library publishes, generated from its compiled environment; a reader that uses an index only after proving, from files alone, that it describes the exact checkout the project builds against; and agent rules that turn a hit into a dependency edge, a one-line reuse, or an explicitly unproved "wanted" result. We deploy it on ATLAS, whose published index holds 11,649 declarations in 2,257 modules. A search over ATLAS answers in about 1.14 s, of which verification takes 366 ms. On 4,005 known-item queries over 801 targets, the reader returns the target every time, and replaying 103 queries that agents actually issued shows both where the design works and where literal, conjunctive matching fails them.

Date: October 15, 2026
Code:
https://github.com/facebookresearch/autoform-bot
https://github.com/facebookresearch/atlas-lean

1 Introduction

Large-scale autoformalization is now a software-engineering problem as much as a proving problem. Systems such as AutoformBot (Rammal et al., 2026) coordinate hundreds of language-model agents that turn textbook mathematics into Lean 4 (de Moura and Ullrich, 2021), and libraries such as ATLAS (ATLAS contributors, 2026) collect their output: tens of thousands of declarations that extend mathlib (The mathlib Community, 2020). Once such libraries exist, a new question dominates the cost of every further project: *is the result I am about to state already somewhere I can use?*

Answering it badly is expensive in three distinct ways. First, a planner that does not know a library proves a theorem schedules work to prove it again; in our trials, an agent without library search wrote its own counting function for sums of three squares although ATLAS already exports one. Second, two parallel agents that each need the same helper produce two declarations of it; both compile, so the proof checker never complains. Third, and most subtly, an answer of the form "no match" is itself a claim. If the search tool silently ignored a library, or read an index that no longer describes the code being built, an agent concludes that the result is new and acts on a false negative.

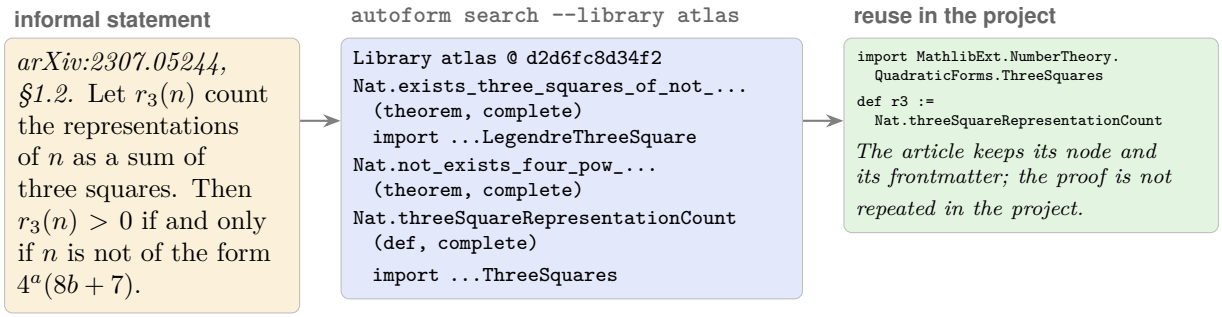


Figure 1 The intended loop, from a development trial. A planner reads a source, searches the blueprint and every usable library before adding a node, and reuses a *complete* hit instead of proving it again. The middle panel abridges the reader's output on the index ATLAS publishes; the right panel is the definition a Formalize agent wrote.

This paper describes the mechanism that AutoformBot uses to search before it adds, and the evidence that it works. Its central design decision is to treat a library index as *untrusted third-party data* that must be bound to the exact checkout a project compiles against before any of it is believed. A second decision is to treat the answer as a contract: every way in which the search cannot speak for a library is a refusal with a reason, never an empty result.

Contributions.

1. **Blueprint search** (§4.1): a read-only, index-free search over Markdown blueprint articles, with one shared definition of an article's statement, Unicode-aware folding, and a ranking that puts the owner of a Lean name ahead of the articles that merely cite it.
2. **A library index and its generator** (§4.2, §4.3): a line-oriented format that records, for every public declaration written by a person, its kind, status (*complete* or *wanted*), statement, signature and docstring, bound to the library's sources by per-module and whole-tree digests; and a generator that reads what only the compiled environment knows through a Lean probe and what only the source knows through a static reader.
3. **A verifying reader** (§4.4): before reading a single declaration, the reader proves from files alone, without running Git, Lake or Lean, that the checkout is at the locked commit, that the pins and every indexed source match, and that the index is one generation.
4. **Distribution and agent protocol** (§4.5, §4.6): a workflow by which ATLAS publishes its index on a tagged commit, a catalog by which new projects require it at that commit, and rules that tell agents what to do with each kind of hit.
5. **Evaluation** (§5) on the published ATLAS index: cost, a known-item retrieval study, a replay of 103 queries issued by agents in 17 sessions, and end-to-end trials.

2 Related Work

Premise selection and retrieval for proving. Choosing which existing facts a proof should use is a classical problem in automated reasoning, addressed with corpus statistics and kernel methods (Alama et al., 2014), learned relevance in hammers (Kaliszyk and Urban, 2014; Blanchette et al., 2016), and neural encoders (Irving et al., 2016; Mikuła et al., 2024). LeanDojo's ReProver (Yang et al., 2023) retrieves premises from mathlib for a language-model prover. These systems answer *which known facts help prove this goal*; we answer a coarser and earlier question, *does a result I am about to plan or state already exist*, and our main concern is not ranking quality but whether a negative answer can be trusted.

Search engines for Lean. Loogle (Breitner, 2023) searches MATHLIB by name and type pattern; LeanSearch (Gao et al., 2024) adds semantic retrieval over informal renderings of MATHLIB declarations. Both index a single, centrally built library. Our setting differs in two ways: the library is a *dependency of the agent's project*, pinned at a commit the project chooses, and the answer must be valid for that exact commit. We therefore ship the index with the library, and the reader verifies it locally rather than querying a service.

Blueprints and autoformalization at scale. Formalization blueprints (Massot, 2023) organize a project as a dependency graph of informal statements linked to Lean declarations. AUTOFORMBOT's blueprint is a vault of Markdown articles in the same spirit (Rammal et al., 2026), and earlier textbook-scale systems (Gloeckle et al., 2026) and autoformalization work (Wu et al., 2022) motivate the scale at which duplication matters. To our knowledge, no prior system lets agents search a blueprint and its library dependencies together before adding a node.

Software supply-chain integrity. Our reader borrows from update and supply-chain security: metadata that is believed only when bound to the artifacts it describes (Samuel et al., 2010; Torres-Arias et al., 2019), and outputs that are a deterministic function of their inputs (Lamb and Zacchiroli, 2022). The index is our "metadata", the library checkout is the "artifact", and the binding is a set of digests the reader recomputes.

Agents and tools. Agent–computer interfaces matter for coding agents (Yang et al., 2024). We find the same in mathematics: whether an agent searches at all, and how it phrases queries, depends more on the skill text and on the shape of the command's output than on the search algorithm (§5.5).

3 Setting and Design Goals

Blueprints. An AUTOFORMBOT project keeps its plan as Markdown files in the directory `blueprint/roadmap/`. Every file is an *article*; a directory's `README.md` contains the articles below it. A formalizable article carries frontmatter (its declaration kind, its `lean:` names, and the formalization state of its statement and proof), an H1 title, a statement, and sections such as `## Depends on` and `## Proof depends on` whose links form the dependency DAG. A scheduler offers an article to a worker only when its prerequisites are in the right state. By default a theorem's statement is accepted only together with its proof (*strict* policy); a project may opt into *open statements*.

Shared libraries. A Lean project depends on other Lake packages through `lakefile.toml` and locks each Git dependency at a commit in `lake-manifest.json`; Lake checks it out under `.lake/packages/<name>`. ATLAS is one such package. It holds three libraries: `MathlibExt` and `CSLibExt` (complete results) and `WantedExt`, whose results are stated with the Batteries (Lean community) commands `theorem_wanted` and `def_wanted`: the statement is checked, the proof is deferred, and the declaration cannot be used by an importer. `WantedExt` also holds complete definitions whose type is `Prop`: they name a statement without proving it.

Threat model. The index is produced in another repository, by a different version of the tooling, at a commit the consumer did not choose. We assume no malice is required for it to be wrong: a stale file, a merge of two regenerations, a checkout moved by hand, an override that points Lake elsewhere, or a library built against a different toolchain are all ordinary. We also treat index text as untrusted input: it is parsed defensively, echoed only in shortened and escaped form, and its prose is data, never instructions to the agent.

Goals.

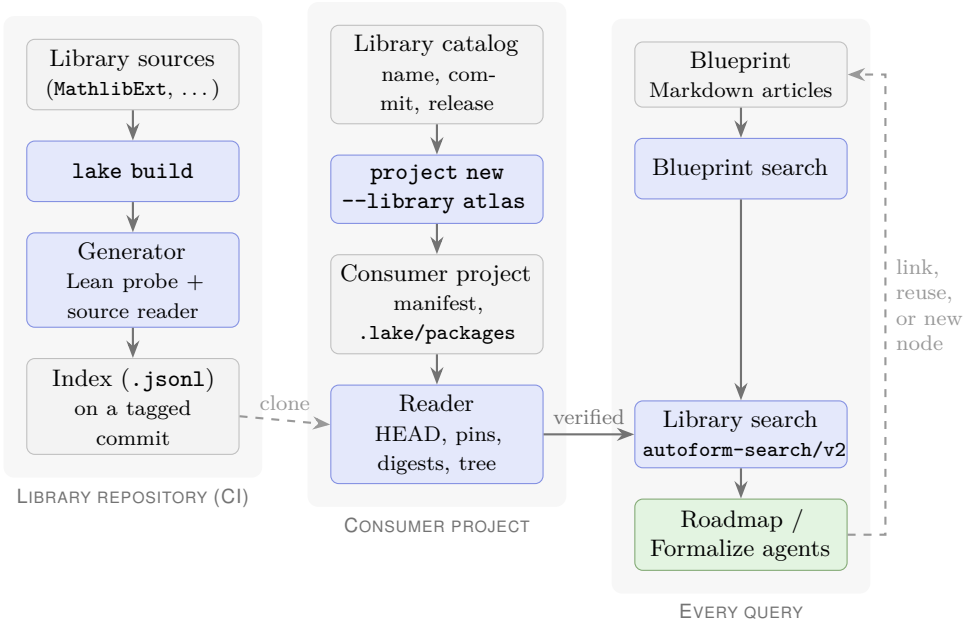


Figure 2 Architecture. The library builds and indexes itself once per tagged commit. A consumer project created from the catalog locks that commit; on each query the reader re-verifies the index against the checkout before the library search reads a declaration. A refusal anywhere refuses the whole call.

G1 No silent negatives. Any reason the search cannot speak for a named library is a refusal with that reason; an empty list means only that nothing matched.

G2 Freshness by construction. A hit describes the code the project will compile; the reader checks this itself rather than trusting the library's CI.

G3 Honest status. *complete* means usable by an importer and resting only on the standard axioms; *wanted* means stated, not proved.

G4 Read-only and bounded. Search and listing start no process, open no socket and write nothing; generation obeys the project's limits on time, output and child processes.

G5 Stable contracts. Output is deterministic JSON under a versioned schema, and the blueprint-only schema is unchanged by library support.

4 The Search-and-Reuse Mechanism

Figure 2 shows the parts and who runs them. The library side runs once per published commit; the consumer side runs on every query.

4.1 Blueprint search

`autoform search TARGET QUERY` returns the articles in which every term of the query occurs. It keeps no index: each call reloads the blueprint graph, reads each article once, and refuses if an article's bytes differ from those the graph was built from, so a hit describes the Markdown as it is.

One definition of a statement. Before search, the audit and the site renderer each decided where an article's statement ends, and disagreed (the audit stopped at the first H2, the site at the first heading of any level). Search would have been a third opinion. We introduced a

single function, `article_parts`, that splits an article into its statement and its H2 sections after masking fenced code and HTML comments; the audit, the renderer and search all use it. Statement text is then judged *as published*: search matches the visible prose the renderer produces, so text in a comment, a code block or the frontmatter never yields a hit. The same change let the audit reject statements that are empty or are only a placeholder (*TODO*, *TBD*), which would otherwise be found by search and read as an existing result.

Fields and ranking. Each article contributes six fields, best first: `title`, `lean` (the last component of each Lean name), `node_id` (the last path component), `statement_text`, `ancestors` (titles of containing articles), and `qualified_names` (full names and path). A query matches an article if every term occurs, as a substring, in some field. Hits are ordered by the best field any term reached, then by whether the words matched as typed (rather than only after stemming), then by the field of the worst-placed term, then by the number of dependents, then by ID. Two choices deserve comment. Indexing only the *last* component of a name in the strong `lean` field prevents a chapter's directory or a project namespace from making every article beneath it a strong match: on a 2,031-article test blueprint, the one article about probability measures ranked 102nd of 102 for *measure* before this change and 2nd after. And a term that spells a whole declaration name or its trailing components (`Convex.separation` of `Project.Convex.separation`) is promoted to a `lean` match, so the owner of a declaration ranks above the articles that cite it.

Folding. Query and fields are folded through NFKD, with accents removed only from letters (a combining mark on a relation negates it, so $\neq$ must not match $=$), invisible characters dropped, typographic dashes and quotes mapped to ASCII, and case folded. Query words shed the punctuation, quotation marks, backticks, dollar signs, emphasis and unbalanced parentheses a sentence or Markdown leaves around them, iterated to a fixed point so that `(**weak**)` and `$L^2$,` reduce to their content. Filler words are dropped, and a word of letters loses one common ending (*-ing, -ed, -es, -s*, with a small set of spelling repairs) in the spirit of suffix stripping (Porter, 1980). Only the query is shortened, so stemming can add hits but never remove one.

Refusals. Search exits with status 2, never with an empty list, when it cannot answer for the whole blueprint: an article changed during the read, a symlinked or escaping path, invalid UTF-8, or a `--declaration` kind that no article in the blueprint uses (which would otherwise make a typo read as "this result is new").

4.2 The library index

An index is a JSON-lines file, `autoform-library-index.jsonl`, at the package root. Its first line is a *header*; then one *module* record per indexed source file; then one *declaration* record per indexed declaration (Appendix A). The header records the generator version, a probe revision, the package name, the indexed source directories, the toolchain, the direct dependencies with their locked revisions, and a *tree digest*. Each module record carries the SHA-256 of its source (with CRLF read as LF). Each declaration records its name, module, written keyword, status, line, printed signature, source statement up to its body, docstring, the constants its type mentions, and whether its name was generated.

The tree digest is a SHA-256 over all module digests in name order followed by the pin files (`lean-toolchain`, `lake-manifest.json`, `lakefile.lean`, `lakefile.toml`) with their lengths. Per-module digests alone are not enough: a declaration's status and printed type depend on other modules, so two indexes that were each current can be merged line by line into one whose every module digest is right but whose declarations are stale. Only regeneration produces a

matching tree digest.

4.3 Generating an index

`autoform library index ROOT` writes the index from the library's current build and never builds. It splits the work by what each side knows.

What the environment knows. A probe, a Lean program rendered from a template, imports every indexed module into one plain (non-module) file and walks the environment. For each constant it reports visibility, the module that declares it, its declaration and selection ranges, its type printed under fixed options (full names, no universes, unbounded width) so that the library's own options and open namespaces cannot change the bytes, its docstring, the constants its type mentions, and its axioms. For *wanted* declarations the probe recognizes the Batteries placeholder types and unwraps them to the stated proposition. The probe runs through AutoformBot's existing bounded runner: an explicit project root, a time limit, an output limit, a freshness check that refuses a stale build, and descendant cleanup. Module names are passed as one string literal read back with `String.toName`; an `Array Name` literal of ATLAS's 2,808 library and test modules exceeded Lean's maximum recursion depth.

What the source knows. The keyword a declaration was written with, the name as written, and the statement text exist only in the source. A pure Python reader skips the docstring, attributes and modifiers in the declaration's range and keeps the declaration only if a declaration keyword follows; it extracts the statement up to the first `:=`, `where` or line-initial `|` outside brackets, comments and strings, treating the `:=` of a `let` or `have` inside the type as part of it. Joining the two sides yields the inclusion rule: a declaration is indexed if it is public, written by a person (its range starts with a keyword and its selection is its name), and either rests only on `propext`, `Classical.choice` and `Quot.sound` (*complete*) or was declared with a wanted command (*wanted*). Constructors, projections, recursors, matchers, generated extensionality lemmas, derived instances, auxiliary definitions and `@[to_additive]` copies are excluded.

Shared names and determinism. Lean accepts two imported modules that declare a theorem with the same name and the same statement and keeps one copy, chosen by load order. The probe reads the copy in the first module in name order, including its own proof's axioms, so the index does not depend on load order. Two modules declaring one name with different statements are refused. Generation is refused if any source changes while the probe runs; the file is written atomically; and every generated index is parsed back by the reader before it is returned. `--check` reports staleness, first from file digests alone so that a forgotten regeneration fails in seconds, then by regenerating and comparing bytes.

4.4 Verifying an index before reading it

`autoform search --library NAME --lean-root ROOT` and `autoform library list` share one routine, `load_library`, which refuses unless every check in Table 1 passes. All checks read files; none runs Git, Lake or Lean. A refusal of any named library refuses the whole call, blueprint hits included, because a partial answer reads as "this result is new".

When the project's own toolchain or dependency revisions differ from those the library was indexed with, the reader does not refuse: Lake builds a dependency with the project's toolchain, so a difference does not mean the library fails to build. It reports each difference beside the hits, and the agent rules say to let a focused build decide.

Table 1 Checks the reader performs, in order, before an index is used.

Check	Refused when
Lock	the package is not in `lake-manifest.json`, is a path dependency, or is replaced by `.lake/package-overrides.json`
Checkout	`.lake/packages/<name>` is missing or reached through a symbolic link
Revision	the checkout's `HEAD`, resolved from loose refs, packed refs, linked worktrees and submodule pointers, is not the locked commit
Format	the index is missing, not a regular file, over 64 MiB, not UTF-8, or outside the schema (unknown keys, repeated keys, `NaN`, lone surrogates, records naming unknown modules)
Pins	a pin file is a link or unreadable; the toolchain or the direct dependencies differ from the header
Sources	a `.lean` file under an indexed directory is not indexed, an indexed file is missing, or a digest differs; a link or special file sits under a source directory
Generation	the recomputed tree digest differs from the header's

Library ranking. Library declarations are matched with the blueprint's folding and term rules over six fields: `lean` (last component), `docstring`, `module_doc`, `qualified_names`, `mentions` and `kind`. Signatures and statements are returned but not matched, because binder words such as *Type* occur in nearly all of them. Among equal matches a complete declaration precedes a wanted one. Each hit carries the `import` line, built from the validated module name; a wanted hit has none, since it cannot be imported for use.

4.5 Publication and distribution

Publishing on a tag. ATLAS merges dozens of pull requests a week, and a committed index on `main` would conflict on its header in every pair of regenerating PRs. We therefore publish an index on a *tagged commit*: a manually triggered workflow builds the three libraries, installs AutoformBot at a pinned revision, runs the generator, checks that the index is the only file the build changed, and, in a second job that alone holds write permission and runs no Lean or package code, commits the index on top of the indexed commit and pushes only the tag `autoform-index/<sha>`. Because the tagged commit adds a single file to the indexed sources, its sources and pins are byte-identical to those the index digests, so the tagged commit verifies.

Requiring a library. A bundled catalog lists each shared library with its Git URL, the full commit of its tag, the toolchain-and-mathlib release it builds with, and the exact manifest entries `lake update` would write. `autoform project new --library atlas` writes, offline and atomically, a project on that release with the library's `require` and a complete lock; it refuses combinations with explicit version options, unknown names, and package names that would shadow the library's module roots. The setup agent offers the library only to a new project and only with the user's consent.

4.6 Agent protocol

Search prevents a duplicate only if the agent runs it and acts on the answer correctly. The planning (Roadmap) and proving (Formalize) skills therefore carry short, tested rules:

- **When.** Before adding a formalizable article, and before stating a result or adding a helper, search the blueprint and every library that `autoform library list` reports as usable, even while the blueprint is empty.
- **How.** Use a few distinctive words and the likely Lean name; since matching is literal and

Table 2 Composition of the published ATLAS index. A declaration is *complete* if it is public, written by a person, and rests only on the three standard axioms; *wanted* if declared with a Batteries wanted command.

Library	Declarations	Complete	Wanted	With docstring
`MathlibExt`	6,852	6,852	0	5,739
`CSLibExt`	2,817	2,817	0	2,435
`WantedExt`	1,980	1,265	715	1,879
Total	**11,649**	**10,934**	**715**	**10,053**

Declarations by written keyword					
`theorem`	6,937	`def`	3,737	`lemma`	320
`abbrev`	208	`instance`	198	`structure`	174
`inductive`	67	`class`	8		

conjunctive, rerun each distinctive word alone before concluding a library lacks a result.

- **Blueprint hit.** Link to it under `## Depends on` instead of adding a node; a more general, special or similar result does not replace the one wanted. In Formalize, a hit's declaration is used only if the selected article's dependencies reach it; otherwise it is a missing prerequisite.
- **Complete library hit.** Keep the article; declare the result under the article's name and prove it from the library, or define a definition as the library's (`def X := Lib.X`), never a copy of its body. A `def` whose type is `Prop` is a statement, not a proof.
- **Wanted library hit.** Formalize the result as the project's policy allows and tell the user the library lists it as unproved.
- **Refusals.** A refusal is never an empty result. A refusal naming a library is not a blocker: rerun without it and report that library's answer as unknown. Dependencies and pins remain the user's decision.

A test reads each rule's paragraph and checks that every instruction is present, and a stderr note reminds an agent that searched without `--library` that the project locks an indexed package.

5 Evaluation

We evaluate on the index that ATLAS published at tag `autoform-index/64c2773ffad3` (commit `d2d6fc8d`), generated on a GitHub runner by AutoformBot 0.5.0. Timings are on an Apple-silicon laptop, best of three, with a consumer project created by `project new --library atlas` and the ATLAS checkout at the locked commit.

5.1 The published index

The index covers 2,257 modules (1,670 with a module header) and 11,649 declarations, 10,934 complete and 715 wanted (Table 2); it is 14.0 MB. 86% of declarations carry a docstring, and 17% of docstrings name a bibliographic source, which the agent rules use to cite the original rather than the library. The index is small relative to the source because most of `MathlibExt`'s declarations are `private` helpers; in the earlier trial run on 1,867 modules, about 22,900 of 29,000 `MathlibExt` declarations were private, and no declaration was left out for resting on `sorry`.

Table 3 Cost of one library search over ATLAS. In-process stages are best of three; end-to-end times include interpreter start-up.

Stage	Time
Read the project's Lake workspace	1 ms
Parse and validate the index (14.0 MB)	216 ms
Walk and hash indexed sources (2,257 files, 41 MB)	154 ms
verification total	*366 ms*
Match one query against 11,649 declarations	606 ms
`autoform library list .` (end to end)	0.50 s
`autoform search ... --library atlas` (end to end)	1.14 s
`autoform search ...` without `--library` (empty blueprint)	0.13 s

Table 4 Known-item retrieval on the ATLAS index with k docstring words per query ($n = 801$ targets per row).

k	Recall	Hit@1	Hit@10	MRR	Median matches
1	100.0%	7.1%	25.3%	0.135	538
2	100.0%	35.0%	75.4%	0.490	42
3	100.0%	64.3%	90.3%	0.743	5
4	100.0%	84.3%	96.3%	0.890	1
5	100.0%	86.3%	96.6%	0.907	1

5.2 Cost

Verification costs 366 ms per call (Table 3), so re-verifying on every query, rather than caching a verdict that could go stale, is affordable. Matching dominates: it re-folds every declaration's fields on each query. Caching the folded fields per verified index is a straightforward optimization we have not needed at this scale. Blueprint search without an index costs about 2 ms per article.

5.3 Known-item retrieval

Protocol. We sample 1,000 declarations with docstrings (fixed seed) and, for each $k \in \{1, \ldots, 5\}$, form a query of k distinct content words (length ≥ 4, not filler) drawn from the declaration's docstring after removing inline code and LaTeX spans. We then rank the whole index and record where the target falls. Declarations whose docstring yields fewer than five such words are skipped, leaving 801 targets per k. Because every query word occurs in the target's docstring, a correct implementation must return the target every time: *recall* here tests the end-to-end folding, bracket peeling and stemming pipeline, while *rank* measures how well the field ordering separates the target from other matches.

Results. Table 4 and Figure 3a show the results. Recall is 100% at every k: folding, bracket peeling and stemming never lose a word that occurs in the target's docstring. Ranking improves quickly with query length, because conjunctive matching shrinks the candidate set: the median number of matches falls from 538 for one word to 1 for five, and Hit@10 rises from 25.3% to 96.6%. Single words are poor discriminators, since docstrings of related declarations share vocabulary, and because docstring matches rank below name matches, a one-word query often surfaces declarations whose *name* contains the word.

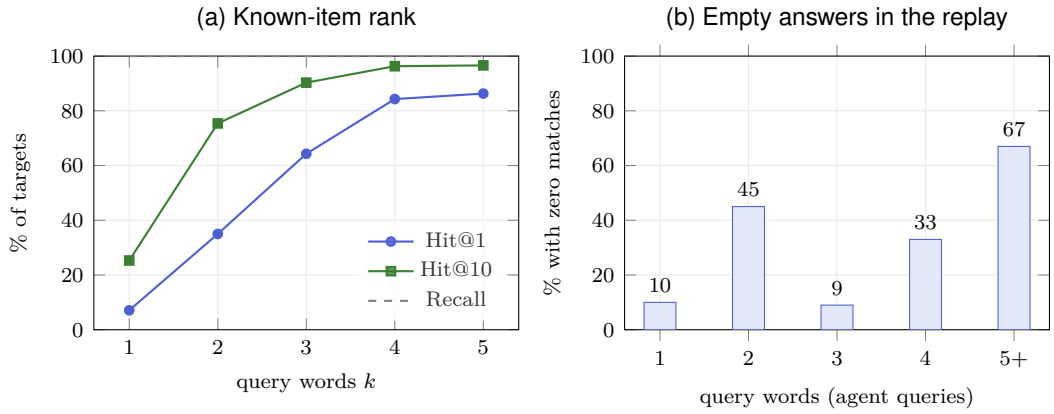


Figure 3 (a) Known-item retrieval on ATLAS as the number of docstring words per query grows ($n = 801$). (b) Share of replayed agent queries with no match, by query length; labels are percentages, and bar heights rest on 103 calls in all (Table 5).

Table 5 Agent queries replayed against the published index, by number of query words after filler removal.

Words	Calls	Zero matches	Median matches
1	21	2 (10%)	8
2	11	5 (45%)	9
3	47	4 (9%)	4
4	18	6 (33%)	1.5
5 or more	6	4 (67%)	0

5.4 Replaying agent queries

Protocol. We extracted every `autoform search` call with `--library atlas` from 17 headless agent sessions run while developing the feature (Roadmap and Formalize on Section 1.2 of arXiv:2307.05244, Gauss's formula for $r_3(n)$), discarding calls whose query was a shell variable. This leaves 103 calls and 60 distinct queries, which we re-run against the published index. The trials ran against earlier test indexes, so absolute counts differ from what the agents saw; the replay measures the current system on realistic queries.

Results. Table 5 shows the cost of conjunctive matching on queries written from a paper rather than from a docstring. One- and three-word queries are empty about one time in ten; four-word queries one time in three; and queries of five or more words 67% of the time. The empty long queries are paraphrases of a theorem's *statement* ("sum of three squares iff", "Legendre Gauss three squares") in which one word, such as *iff* or *Gauss*, occurs nowhere in the target's fields. The rule to rerun each distinctive word alone exists for exactly this reason.

The design nonetheless reaches what the task needs. Five ATLAS declarations mattered in these trials: the counting function `Nat.threeSquareRepresentationCount`, both directions of Legendre's three-square theorem, the Hurwitz class number (*wanted*), and Gauss-reduced binary quadratic forms. Each ranks first for at least 5 of the 60 distinct queries.

Failure modes. The replay exposes three defects worth fixing:

- **Hyphenated compounds.** *three-square representation count* keeps *three-square* as one term, which no camel-case Lean name contains; it returns 2 matches and misses the definition that the unhyphenated query ranks first.
- **Short-term substrings.** *r3* matches inside `betaChar3` and `Collinear3` as a strong `lean`

Table 6 Headless agent trials during development (Claude Code, one session per skill). "Before" and "after" are the same prompts without and with the change named in the first column.

Change	Before	After
Library search and skills	Formalize wrote its own r_3 and never mentioned ATLAS	Roadmap searched ATLAS; Formalize defined r_3 as `Nat.threeSquare RepresentationCount`
Inline library rule in Roadmap	Roadmap searched ATLAS in 0 of 2 runs	4 of 4 runs (4, 7, 8, 9 library searches); 5 complete declarations reused, 3 recorded as wanted
Launcher stated in each skill	Roadmap guessed how to run the CLI in 8 of 8 sessions, Formalize in 5 of 5; two Roadmap sessions and one Formalize session never reached it	0 guesses; Roadmap ran 8 ATLAS searches and reused 3 results; Formalize proved the Legendre corollary from ATLAS lemmas

match and ranks them first among 113 matches.

- **Proper names.** *Legendre three squares* ranks `r3_legendre_recursion` above Legendre's theorem, whose name does not contain *Legendre*; the theorem is second.

5.5 End-to-end trials

Table 6 summarizes the headless trials recorded during development, each a fresh chain of Setup, Roadmap and Formalize sessions with the plugin loaded from the branch under test. The numbers are small and come from the pull-request records, so we read them as evidence of mechanism rather than as effect sizes.

Three observations stand out. First, retrieval quality was never the binding constraint; *invocation* was. The agents did not search when the library rule lived in a linked page, and could not search when the command's launcher was not stated where they read. Second, the agents acted on the fields the output shows: they added the `import` line a complete hit prints, and reported *wanted* hits to the user as unproved, as the rules ask. Third, the remaining failure is in planning, not search: in one of two finished runs, Roadmap found both directions of Legendre's theorem in ATLAS yet copied the paper's proof route through an unproved theorem into `## Proof depends on`, which kept the article blocked under the strict policy.

5.6 Testing

The mechanism is backed by 265 test functions across nine test files (many parametrized), plus 38 skill-text tests. A real-Lean fixture library on Lean 4.34.1 that depends on Batteries at the revision ATLAS locks exercises the generator end to end: every excluded kind, the four wanted commands, `_root_` and quoted names, the two duplicate-name cases, a timeout, and a consumer that imports a complete hit and cannot resolve a wanted one. The reader's refusals were checked with single-line mutations.

6 Discussion and Limitations

What a "complete" status rests on. The probe takes a declaration's axioms from Lean's `collectAxioms`. A separate audit in AutoformBot found that, on an earlier Lean release, `collectAxioms` can omit an axiom reached through the second type of a mutual inductive block, and replaced it with its own walk; the library probe has not yet adopted that walk. A theorem

could therefore be indexed as complete while resting on `sorry` in that narrow case. Similarly, the tree digest covers only the indexed source directories: an indexed module that imports an *unindexed* module of the same package could go stale undetected. ATLAS's indexed libraries import no such module today, but the generator should refuse the configuration rather than rely on it.

What the reader binds and what it does not. The reader ties every source file to its digest, but not a module record's *name* to its file, so a hand-edited index could print a wrong `import` line; a wrong import fails loudly at build time. It does not detect a package replaced with `lake --packages=`, nor edits to declaration records that leave the tree digest intact. These are bounded by the library publishing with `--check` in its own CI, but they are gaps against goal G2.

Matching. Substring, conjunctive matching is predictable and cheap, and its negatives are easy to explain to an agent. Its costs are the vocabulary mismatches of §5.4. Splitting hyphenated and camel-case compounds, requiring whole-token matches for very short terms, and matching *theorem* with *lemma* are low-risk fixes. Semantic retrieval (Gao et al., 2024) would help paraphrase, but would also make an empty answer harder to interpret, which is the property G1 protects.

Operational dependencies. The generator is verified only on Lean 4.34.1 and refuses other toolchains, because the probe reads Lean internals whose meaning can change between releases; a toolchain bump in ATLAS pauses publication until AutoformBot is re-verified. The index format is still `v0`. Consumers pin a commit that only the `autoform-index/*` tag keeps reachable, so the tag must be protected from deletion. The pinned index also ages: the catalog pins one commit, and nothing yet decides when to publish again and move the pin.

Noise in the listing. `autoform library list` reports every locked package, including mathlib and its inherited dependencies, none of which publish an index. The agent rule to report each unusable package as "unknown" then produces the same ten notices in every session. Restricting the report to direct dependencies, and treating mathlib through its own checkout search, would remove it.

7 Conclusion

Reuse in multi-agent autoformalization needs more than a search engine. It needs an answer that is valid for the exact code a project builds, a status that says honestly whether a result can be used, and rules that turn each kind of hit into the right change to the plan. AutoformBot's mechanism provides these with a read-only blueprint search, a library index bound to its sources by digests, a reader that re-verifies it on every query in a few hundred milliseconds, and a publication path that ATLAS now uses. The evaluation suggests where the next gains lie: not in better ranking, but in matching that tolerates the way agents paraphrase mathematics, and in planners that drop a source's proof route once a library has made it unnecessary.

Acknowledgments. The pull requests behind this work were reviewed by the AutoformBot maintainers; we thank Deicyde for detailed reviews of the library reader and generator, and toskua for an independent end-to-end trial.

References

Jesse Alama, Tom Heskes, Daniel Kühlwein, Evgeni Tsivtsivadze, and Josef Urban. Premise selection for mathematics by corpus analysis and kernel methods. *Journal of Automated Reasoning*, 52(2):191–213, 2014.

ATLAS contributors. ATLAS: Autoformalized textbook library at scale. https://github.com/facebookresearch/atlas-lean, 2026.

Jasmin Christian Blanchette, Cezary Kaliszyk, Lawrence C. Paulson, and Josef Urban. Hammering towards QED. *Journal of Formalized Reasoning*, 9(1):101–148, 2016.

Joachim Breitner. Loogle: Lean search engine. https://loogle.lean-lang.org, 2023.

Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.

Guoxiong Gao, Haocheng Ju, Jiedong Jiang, Zihan Qin, and Bin Dong. A semantic search engine for Mathlib4, 2024.

Fabian Gloeckle, Ahmad Rammal, Charles Arnal, Remi Munos, Vivien Cabannes, Gabriel Synnaeve, and Amaury Hayat. Automatic textbook formalization, 2026.

Geoffrey Irving, Christian Szegedy, Alexander A. Alemi, Niklas Eén, François Chollet, and Josef Urban. DeepMath – deep sequence models for premise selection. In *Advances in Neural Information Processing Systems (NIPS)*, 2016.

Cezary Kaliszyk and Josef Urban. Learning-assisted automated reasoning with Flyspeck. *Journal of Automated Reasoning*, 53(2):173–213, 2014.

Chris Lamb and Stefano Zacchiroli. Reproducible builds: Increasing the integrity of software supply chains. *IEEE Software*, 39(2):62–70, 2022.

Lean community. Batteries: the "batteries included" extended library for Lean 4. https://github.com/leanprover-community/batteries. Revision `f2effa3`.

Patrick Massot. leanblueprint: a plasTeX plugin for formalization blueprints. https://github.com/PatrickMassot/leanblueprint, 2023.

Maciej Mikuła, Szymon Tworkowski, Szymon Antoniak, Bartosz Piotrowski, Albert Q. Jiang, Jin Peng Zhou, Christian Szegedy, Łukasz Kuciński, Piotr Miłoś, and Yuhuai Wu. Magnushammer: A transformer-based approach to premise selection. In *International Conference on Learning Representations (ICLR)*, 2024.

Martin F. Porter. An algorithm for suffix stripping. *Program*, 14(3):130–137, 1980.

Ahmad Rammal, Niket Patel, Fabian Gloeckle, Amaury Hayat, Julia Kempe, Remi Munos, Charles Arnal, and Vivien Cabannes. Formalizing mathematics at scale, 2026.

Justin Samuel, Nick Mathewson, Justin Cappos, and Roger Dingledine. Survivable key compromise in software update systems. In *Proceedings of the 17th ACM Conference on Computer and Communications Security (CCS)*, pages 61–72, 2010.

The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, pages 367–381, 2020.

Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. In *28th USENIX Security Symposium*, pages 1393–1410, 2019.

Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

A Index Records

Each line is one JSON object with sorted keys and no insignificant white space. The reader refuses any key outside these sets and any value outside its type.

Record	Field	Meaning
`header`	`schema`	`autoform-library-index/v0`
	`generator`	installed AutoformBot version that wrote the file
	`probe`	integer raised by any change that can alter index bytes
	`package`	the package's own name, from its manifest or lakefile
	`source_dirs`	one entry per library root, e.g. `MathlibExt`
	`lean_toolchain`	as `lean-toolchain` pins it
	`dependencies`	direct dependencies: name, `git`/`path`, locked commit
	`tree`	SHA-256 over module digests and pin files
	`record`	`"header"`
`module`	`name`	Lean module name (ASCII identifier components)
	`source_file`	path under a source directory
	`sha256`	digest of the source with CRLF read as LF
	`module_doc`	the module header, if any
	`module_system`	whether the file is a Lean module file
`declaration`	`name`, `module`	full name and declaring module (the record key)
	`kind`	written keyword: `theorem`, `lemma`, `def`, `abbrev`, `instance`, `structure`, `class`, `inductive`, `opaque`
	`status`	`complete` or `wanted`
	`line`	line of the first modifier or keyword
	`signature`	type printed under fixed options
	`statement`	source from the first modifier to the start of the body
	`docstring`	as Lean records it
	`mentions`	constants the type uses
	`auto_named`	whether the name was generated (anonymous instance)

B Command-Line Contracts

```
autoform search TARGET QUERY [--lean-root ROOT] [--library NAME[=PATH]]...
                [--state KEY]... [--declaration KIND]... [--limit N] [--json]
autoform library list ROOT [--json]
autoform library index ROOT [--lib NAME]... [--check] [--timeout SECONDS]
autoform project new DIR --package NAME --library NAME [--json]
```

Without `--library`, `search --json` emits `autoform-search/v1`, byte-identical to its output before library support (checked on the bundled example over 12 query variants). With `--library` it emits `autoform-search/v2`, which keeps every `v1` key and adds `libraries`. `library list --json` emits `autoform-library-list/v1`. Exit status is 0 for an answer, 2 for a refusal; `library index --check` exits 1 for a stale index.

C Development History

The mechanism landed in six pull requests to AutoformBot and one to ATLAS. Most went through several review rounds before merging.

PR	Merged	Lines added	Content
AutoformBot #171	Oct 8, 2026	2,530	one statement definition; placeholder audit; `autoform search`; search-before-add rules
AutoformBot #211	Oct 12, 2026	221	the Lean 4.34.1 release; a Setup rule for library pins
AutoformBot #204	Oct 13, 2026	7,046	library reader, index generator and Lean probe; library search rules
AutoformBot #311	Oct 14, 2026	720	library catalog; `project new --library`; inline Roadmap rule; stderr note
AutoformBot #319	Oct 14, 2026	65	the CLI launcher stated in every CLI-running skill
ATLAS #164	Oct 15, 2026	162	manual workflow that publishes the index on a tag
AutoformBot #343	Oct 15, 2026	14	catalog points at the published upstream index and offers it